



Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Estado de México

Revisión 3: Avance al 60%

Integrantes:

Carlos Antonio Tejero Andrade A01801062

Luis Angel Godinez Gonzalez A01752310

Eduardo Serrano Corona A01749697

TC2008B.302 Modelación de sistemas multiagentes con gráficas computacionales

Equipo: 2

Profesores:

Maestro. Jorge Adolfo Ramirez Uresti

Maestro. Mauricio Bezares Peñuñuri

Fecha de entrega:

Jueves 27 de Noviembre de 2025

Descripción del Medio Ambiente (Environment)

El entorno de simulación representa una intersección urbana modelada mediante una cuadrícula discreta (*grid*). Basándonos en la clasificación estándar de Russell & Norvig, el entorno posee las siguientes características:

1. **Accesibilidad: Parcialmente Observable (Accesible: 75% No Accesible: 25%).**

Justificación: Cada agente tiene un acceso parcial de la información del entorno limitado solo a sus proximidades. No puede ver el estado de los semáforos de las calles transversales ni la ubicación de vehículos que no estén en su línea de visión directa.

2. **Determinismo: (Determinista: 40% No determinista: 60%).**

Justificación: Aunque las reglas de movimiento son fijas, el entorno contiene incertidumbre. La llegada de nuevos vehículos a la intersección y sus tiempos exactos de solicitud (ETA) no son predecibles por un solo agente, introduciendo variabilidad en la ejecución.

3. **Dinámica: (Dinámico: 80% No Dinámico: 20%).**

Justificación: El entorno cambia de estado continuamente mientras el agente delibera. Los semáforos cambian de color y otros vehículos se mueven independientemente de las acciones del propio agente.

4. **Naturaleza: (Discreto: 100% Continuó: 0%).**

Justificación: Tanto el espacio (celdas de la cuadrícula) como el tiempo (pasos o *steps* de la simulación) son finitos y contables, e inclusive los agentes se encuentran en un mapa con un mapa de tamaño limitado. Las decisiones se toman en intervalos discretos.

5. **Episodicidad: (No episódico: 100% Episódico: 0%).**

Justificación: Las decisiones actuales de un agente (ej. acelerar o frenar) afectan drásticamente sus decisiones futuras. No son episodios aislados; es una secuencia continua de estados.

Definición de Agentes (PEAS)

Para resolver el reto de la intersección inteligente, se definen dos tipos de agentes con roles específicos.

Agente: Vehículo Inteligente

Tipo de Agente: Agente Basado en Objetivos (*Goal-Based Agent*).

- *Razón:* El vehículo no solo reacciona a la luz; tiene un objetivo explícito (cruzar la intersección y llegar a un destino) y planifica sus acciones (calcular ETA) para cumplirlo.

Análisis PEAS:

- **P (Performance / Medida de Desempeño):**
 - Seguridad (0 colisiones con autos y peatones).
 - Legalidad (Respetar señales de luz roja).
- **E (Environment / Medio Ambiente):**
 - Celdas de la calle (Grid), Semáforo asignado a su carril, Vehículos precedentes.
- **A (Actuators / Actuadores):**
 - Ruedas para avanzar una celda.
 - Frenos para mantener la posición.
- **S (Sensors / Sensores):**
 - Sensor_Visual: Detecta el color del semáforo y obstáculos frontales, para evitar colisiones con otros vehículos.
 - GPS/Odometría: Conoce su distancia a la intersección para calcular el ETA (Estimated Time of Arrival).

Agente: Semáforo Coordinador

Tipo de Agente: Agente Reactivo con Estado Interno y Comunicación.

- *Razón:* Debe mantener un estado interno (saber su condición actual, para después cambiarla a una diferente después de un tiempo).

Análisis PEAS:

- **P (Performance / Medida de Desempeño):**
 - Fluidez (Mantener un intercambio de señales constante).
 - Seguridad (Evitar conflictos: nunca poner dos verdes perpendiculares).
- **E (Environment / Medio Ambiente):**
 - Intersección (los 4 carriles), Red de semáforos vecinos.
- **A (Actuators / Actuadores):**
 - Las luces del propio semaforo para allternar entre Verde, Amarillo, Rojo.
- **S (Sensors / Sensores):**
 - Receptor_V2I: Recibe solicitudes y ETAs de los vehículos.

Agente: Peatón

Tipo de Agente: Agente Reactivo

- *Razón:* El peatón responde al estímulo inmediatos(luz peatonal, presencia de vehículos) sin planificar objetivos complejos. Su comportamiento es inmediato, cruza o espera.

Análisis PEAS:

- **P (Performance / Medida de Desempeño):**
 - Seguridad personal (cruzar sin riesgo)
 - Respeto a la señal personal
 - Evita conflictos con vehículos
- **E (Environment / Medio Ambiente):**
 - Banqueta y cruce peatonal
 - Semáforo personal (luz Walk/luz Dont Walk)
 - Otros peatones
- **A (Actuators / Actuadores):**
 - Piernas para avanzar una celda hacia el otro lado, hacia enfrente, o esperar en la banqueta.
 - Ojos para detectar la señal del semáforo para detectar si su luz está en verde o rojo, y decidir si avanzar o quedarse en la banqueta
- **S (Sensors / Sensores):**
 - Sensor_LuzPeatonal: Detecta luz verde/roja para peatones
 - Sensor_ProximidadVehiculo: Percibe vehículos muy cercanos
 - PercepciondelEntorno: Observa a otros peatones para evitar coches

Diagramas de Agentes:

Agente: Semáforo
Grupo: Infraestructura Urbana
Rol: Controlador de tráfico en intersección
Servicios: Informar sobre el estado del semáforo a vehículos y peatón
Protocolos: Comunicación con Vehículos (recibir solicitud)
Eventos: Recepción de solicitudCruce(ETA), Finalización de un ciclo de verde o rojo
Metas: Dar vialidad, e información sobre su estado actual a otros agentes (vehículo y peatón)
Planes: Adaptar el tiempo de cambio
Acciones: Cambiar luz(Color), Enviar ordenBloqueo(ROJO) a vecinos, Enviar liberarBloqueo(AMARILLO) a vecinos, Enviar ordenPaso(VERDE) a vecinos
Conocimientos: Estado actual de su luz

Agente: Peatón
Grupo: Ciudadanos
Rol: Usuarios del sistema vial
Servicios: Desplazarse de forma segura a pie
Protocolos: Comunicación con semáforo peatonal (para un cruce seguro)
Eventos: Detección del cambio de luz del semáforo, detección de vehículo cercano
Metas: Cruzar calles de manera segura y eficiente
Planes: Esperar luz verde -> cruzar -> continuar caminando
Acciones: Solicitar cruce, esperar, y cruzar
Conocimientos: Ubicación actual, estado del semáforo, entorno inmediato

Agente: Vehículo
Grupo: Transporte Particular
Rol: Usuario de la vía, Busca estacionamiento
Servicios: Desplazarse de forma segura a su destino (estacionamiento)
Protocolos: Comunicación con Semáforo (solicitar confirmación cruce)
Eventos: Detección de semáforo cercano, Detección de obstáculo (otro vehículo)
Metas: Llegar al estacionamiento asignado, Cruzar intersecciones de manera segura y legal
Planes: Seguir ruta → Respetar semáforos → Evitar colisiones → Estacionar
Acciones: Avanzar, Frenar, Girar, Enviar solicitudCruce(ETA), Recibe Confirmacion()
Conocimientos: Ubicación actual, Destino, Ruta asignada, Estado del semáforo asignado

Diagrama Organización SMA:

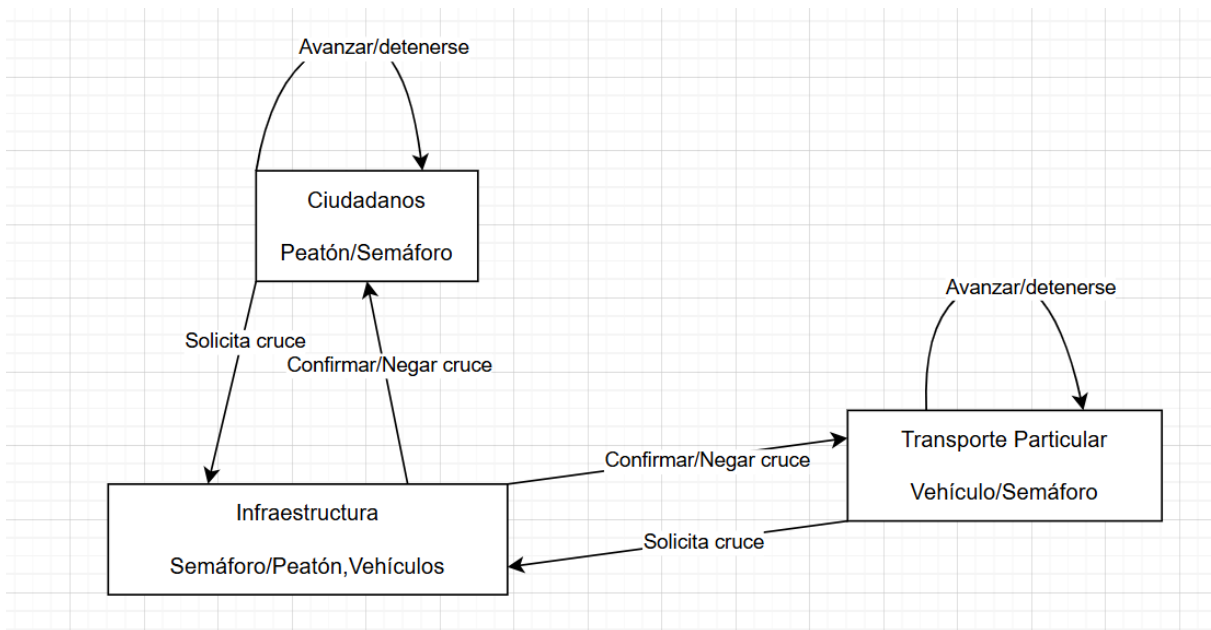
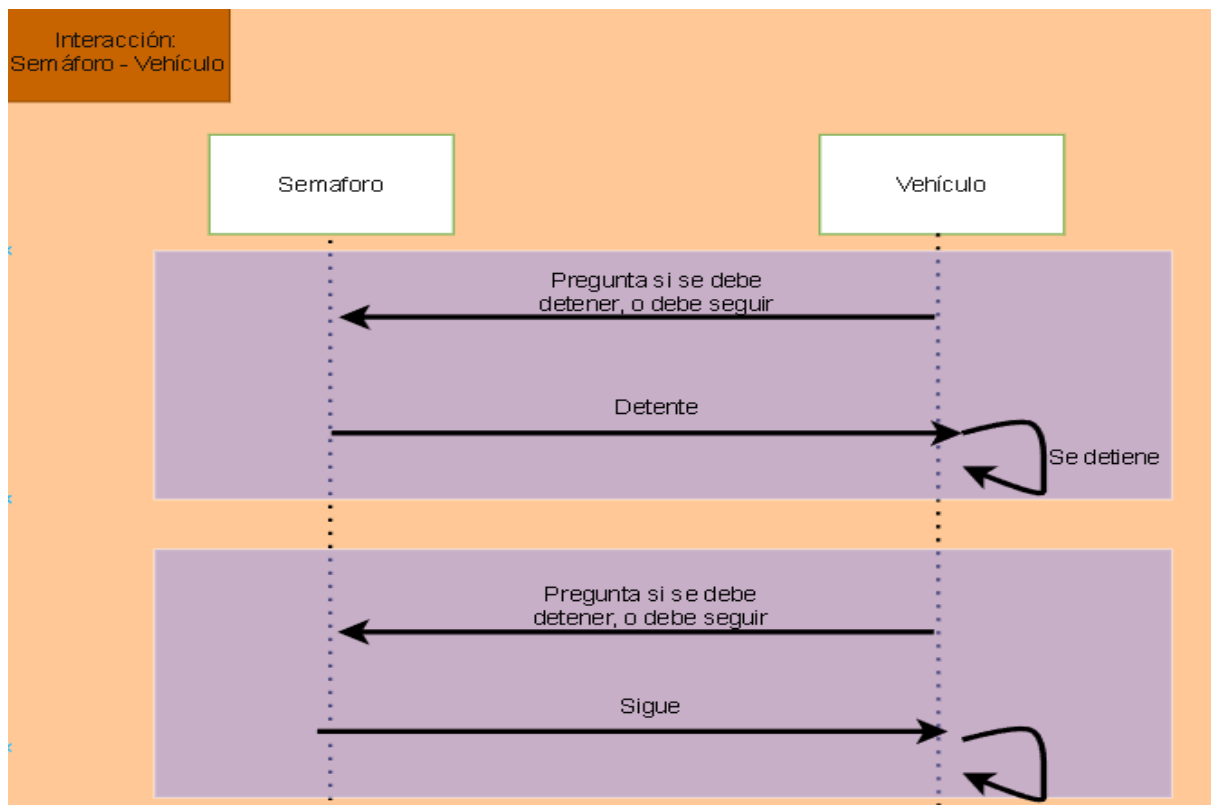
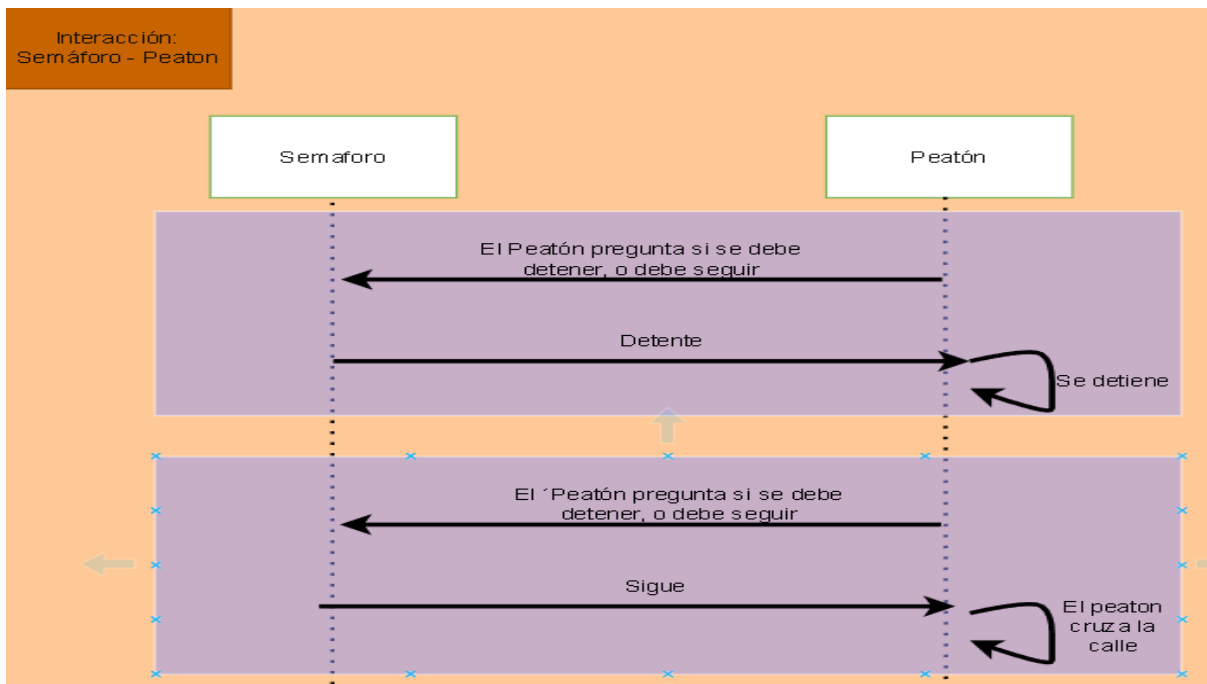


Diagrama SMA:





Protocolo de Interacción (Lógica del Sistema)

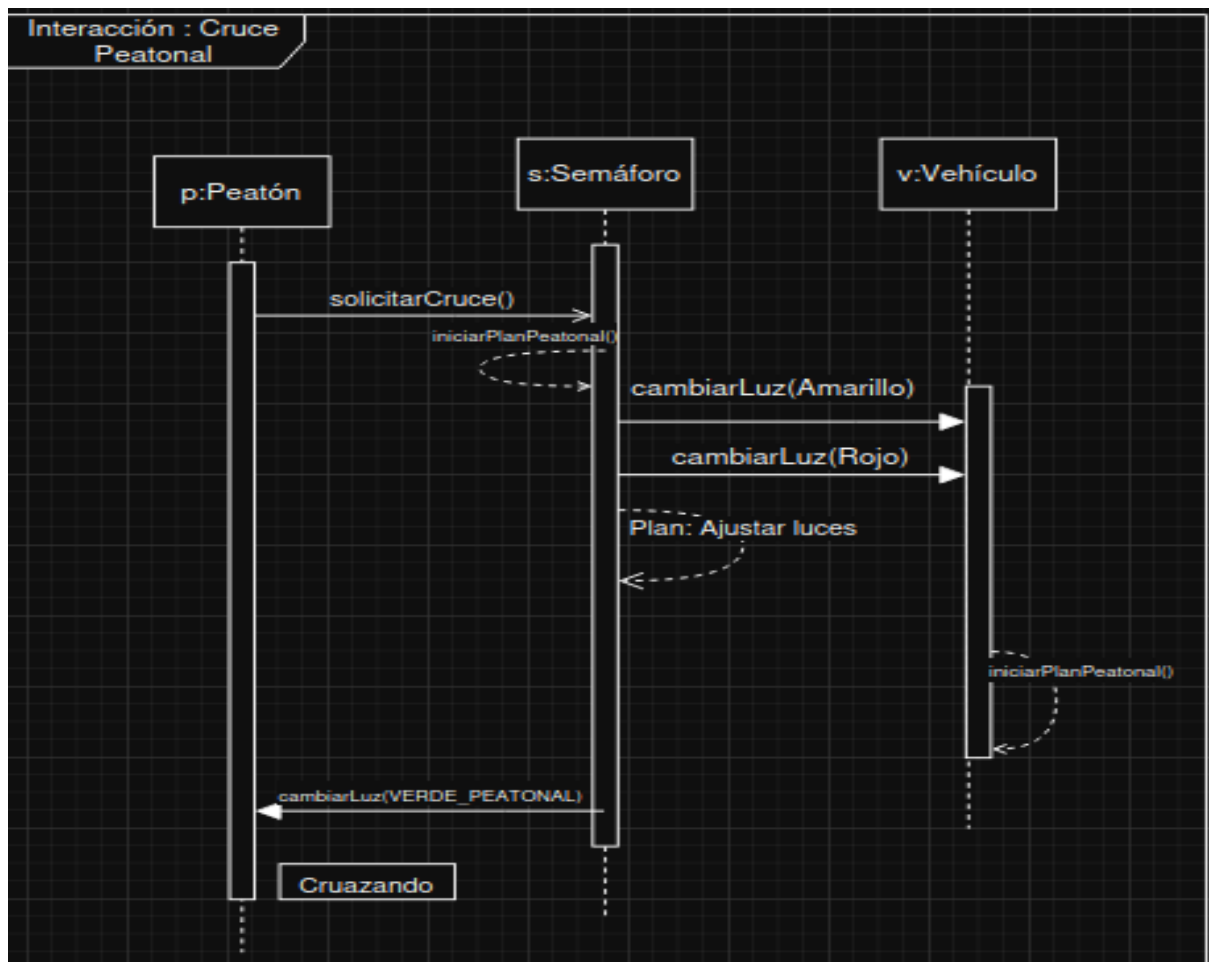


Figura 1. Diagrama de Interacción del sistema

El sistema opera bajo un protocolo de "Demanda y Coordinación". A diferencia de los semáforos tradicionales de tiempo fijo, este sistema prioriza el flujo dinámico.

Estados Posibles del Semáforo:

1. AMARILLO (Idle/Reposo): Estado por defecto cuando no hay vehículos. Indica precaución.
2. ROJO (Bloqueo): Estado activo cuando otra vía tiene el paso.
3. VERDE (Paso): Estado activo otorgado bajo demanda.

Secuencia de Interacción:

Fase 1: Aproximación y Solicitud

1. El Semáforo se encuentra en estado AMARILLO (sin tráfico).
2. Un vehículo entra al rango de detección de la intersección.
3. El Vehículo calcula su ETA (*Tiempo Estimado de Arribo*) basándose en su velocidad y distancia.
4. El Vehículo envía un mensaje al Semáforo de su carril: **SOLICITUD_CRUCE** {ETA: 5s, ID: "Auto_01"}.

Fase 2: Coordinación y Arbitraje 5. El Semáforo Receptor recibe la solicitud. * *Si la intersección está libre*: Inicia el protocolo de bloqueo. * *Si la intersección está ocupada*: Pone la solicitud en cola. 6. El Semáforo Receptor envía un mensaje a los otros 3 semáforos de la intersección: **ORDEN_BLOQUEO**. 7. Los Semáforos Vecinos cambian inmediatamente a ROJO y confirman: **BLOQUEO_CONFIRMADO**.

Fase 3: Ejecución 8. Al recibir las confirmaciones, el Semáforo Receptor cambia a VERDE. 9. El Vehículo cruza la intersección de manera segura.

Fase 4: Retorno al Reposo 10. Una vez que el vehículo ha cruzado (o expirado el tiempo del ETA + margen), el Semáforo Receptor envía la señal **LIBERAR_INTERSECCION**. 11. Todos los semáforos regresan a su estado base: **AMARILLO**, esperando la siguiente solicitud.

Plan de Trabajo y Aprendizaje Adquirido:

Objetivo	Tarea Específica	Deadline (Fecha Límite)	Responsable(s)	Esfuerzo Estimado	Tiempo que tomó realizarlas
1. Documentación y Diseño (Entregas Inmediatas) Finalizado	Revisión 1: Finalizar Diagramas de Secuencia y AUML (Agentes).	13 de Noviembre	Luis Ángel Godínez, y Eduardo Serrano Corona	2 - 3 horas	4 horas
Finalizado	Revisión 2: Redacción del Análisis PEAS, Medio Ambiente y Protocolo de Intersección.	14 Noviembre	Luis Ángel Godínez	2 horas	3 horas
2. Lógica del Sistema (Backend - Python/Mesa) Finalizado	Programación del Grid y movimiento básico de vehículos (sin colisiones).	15 Noviembre	Luis Ángel Godínez	4 - 5 horas	4 horas
Finalizado	Implementación de la lógica de "Semáforo Inteligente" (Amarillo por defecto, cambio a demanda).	18 Noviembre	Luis Ángel Godínez	6 - 8 horas	7 horas
Finalizado	Configuración del Servidor TCP/Socket en	20 Noviembre	Luis Ángel Godínez	3 horas	2 horas

	Python para enviar datos.				
3. Visualización y Entorno (Frontend - Unity) Finalizado	Modelado/Búsqueda de Assets 3D (Coches, Semáforos con 3 luces, Edificios).	21 Noviembre	Eduardo Serrano	4 - 5 horas	6 horas
Finalizado	Construcción de la Escena (Grid 3D) basada en el mapa proporcionado (Calles y Estacionamientos).	22 Noviembre	Carlos Tejero y Eduardo Serrano	5 - 6 horas	7 horas
Finalizado	Scripting C#: Cliente TCP para recibir coordenadas desde Python.	24 Noviembre	Carlos Tejero y Eduardo Serrano	6 - 8 horas	7 horas
4. Integración y Sincronización En progreso	Revisión 3: Corrección del Análisis de Peas, y e la descripción del medio ambiente	27 Noviembre	Eduardo Serrano	2 horas	3 horas
En progreso	Conexión Backend-Frontend: Lograr que el cubo en Unity se mueva igual que en Python.	28 Noviembre	Todos (Lidera Luis la lógica)	5 horas (Sesión conjunta)	

En progreso	Visualización de estados del Semáforo (Cambio de materiales/luces en Unity).	29 Noviembre	Eduardo Serrano	3 horas	
5. Pruebas y Entrega Final En progreso	Depuración: Verificar que los autos no choquen y respeten la lógica de "pide y pasa".	30 Noviembre	Todos	4 - 6 horas	
En progreso	Grabación de evidencia y preparación de presentación final.	02 Diciembre	Carlos Tejero	2 horas	

Aprendizaje Adquirido

1. Modelado de Sistemas Complejos: Hemos aprendido a descomponer un problema macro (tráfico urbano) en micro-comportamientos autónomos. Comprendimos que la complejidad del tráfico no se programa con un "controlador central gigante", sino que emerge de la interacción local entre agentes simples (Vehículos y Semáforos).

2. Arquitectura de Agentes BDI y Reactivos: Diferenciamos en la práctica entre agentes **Deliberativos** (Vehículos que tienen metas, calculan rutas y tiempos ETA) y agentes **Reactivos/Coordinadores** (Semáforos que reaccionan a estímulos y coordinan su estado con otros pares para mantener la seguridad).

3. Protocolos de Comunicación Distribuidos: Implementamos un sistema de comunicación dual:

- **V2I (Vehicle-to-Infrastructure):** Del coche al semáforo para solicitar el paso.
- **I2I (Infrastructure-to-Infrastructure):** Entre semáforos para bloquear intersecciones. Esto nos enseñó la importancia de la sincronización y el paso de mensajes para evitar condiciones de carrera (choques) en sistemas concurrentes.

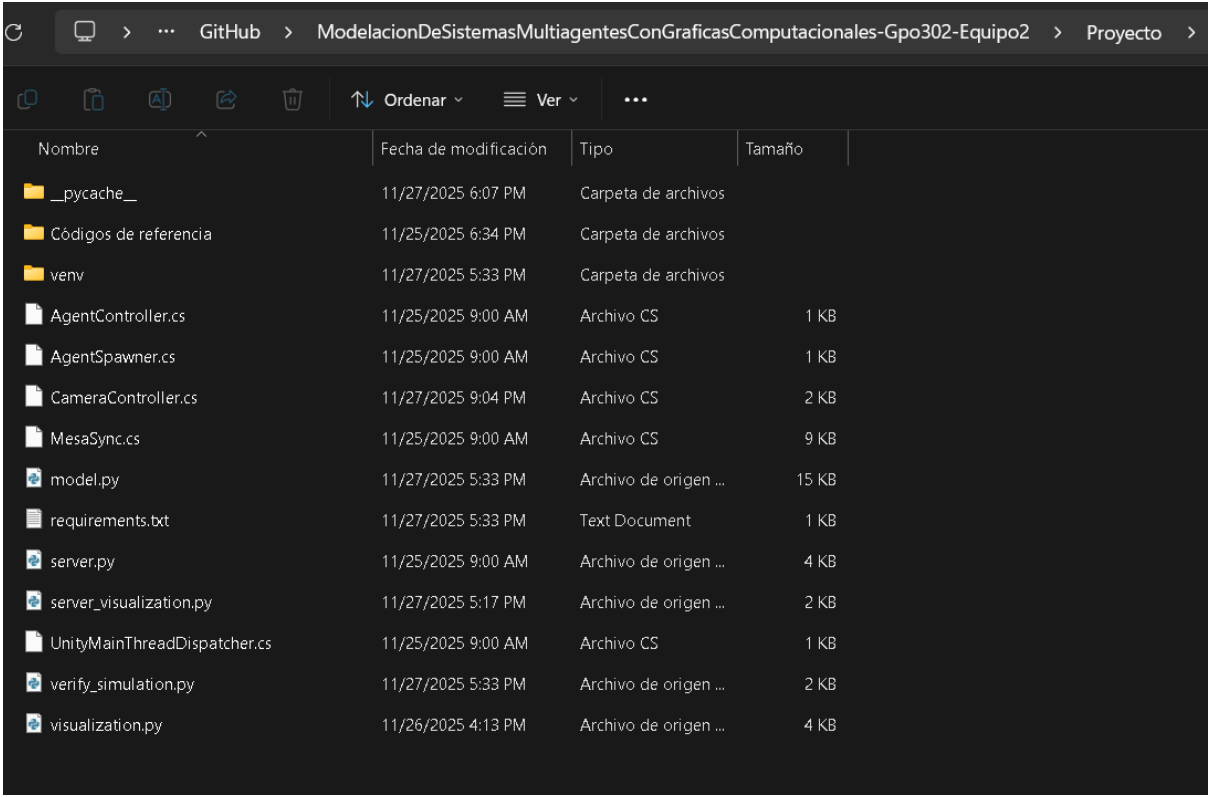
4. Integración Inter-Plataformas: Adquirimos la competencia técnica para conectar dos entornos distintos: un motor de simulación lógica basado en datos (**Python/Mesa**) y un motor

de renderizado gráfico en tiempo real (**Unity**), comprendiendo el flujo de datos a través de sockets TCP/IP.

5. Uso de la librería mesa: Implementamos la librería mesa para la actividad integradora, la cuál fue utilizada para la simulación y prueba de los agentes del proyecto, haciendo uso del visualizador de esta misma librería para ello, además del armado de los propios agentes.

6. Modelado en 3d para Unity: Para esta etapa de avance del 60%, ya debíamos tener hecho el grid del mapa donde se hará la simulación final y los modelos de los agentes implicados, usamos las herramientas de la pestaña ProBuilder del editor, las cuales permiten modificar figuras de manera más exhaustiva, lo que permitió generar los assets.

Código (al menos el 60%) de la implementación de los agentes:



The screenshot shows a file explorer window with the following path: GitHub > ModelacionDeSistemasMultiagentesConGraficasComputacionales-Gpo302-Equipo2 > Proyecto. The file list is as follows:

Nombre	Fecha de modificación	Tipo	Tamaño
__pycache__	11/27/2025 6:07 PM	Carpeta de archivos	
Códigos de referencia	11/25/2025 6:34 PM	Carpeta de archivos	
venv	11/27/2025 5:33 PM	Carpeta de archivos	
AgentController.cs	11/25/2025 9:00 AM	Archivo CS	1 KB
AgentSpawner.cs	11/25/2025 9:00 AM	Archivo CS	1 KB
CameraController.cs	11/27/2025 9:04 PM	Archivo CS	2 KB
MesaSync.cs	11/25/2025 9:00 AM	Archivo CS	9 KB
model.py	11/27/2025 5:33 PM	Archivo de origen ...	15 KB
requirements.txt	11/27/2025 5:33 PM	Text Document	1 KB
server.py	11/25/2025 9:00 AM	Archivo de origen ...	4 KB
server_visualization.py	11/27/2025 5:17 PM	Archivo de origen ...	2 KB
UnityMainThreadDispatcher.cs	11/25/2025 9:00 AM	Archivo CS	1 KB
verify_simulation.py	11/27/2025 5:33 PM	Archivo de origen ...	2 KB
visualization.py	11/26/2025 4:13 PM	Archivo de origen ...	4 KB

Código (al menos el 60%) de la implementación de la parte gráfica de la solución:

ModelacionDeSistemasMultiagentesConGraficasComputacionales-Gpo302-Equipo2 > RacoonCity > Assets > Scripts				
Ordenar Ver				
Nombre	Fecha de modificación	Tipo	Tamaño	
AgentController.cs	11/25/2025 9:00 AM	Archivo CS	1 KB	
AgentController.cs.meta	11/25/2025 9:00 AM	Archivo META	1 KB	
AgentSpawner.cs	11/25/2025 9:00 AM	Archivo CS	1 KB	
AgentSpawner.cs.meta	11/25/2025 9:00 AM	Archivo META	1 KB	
CameraController.cs	11/27/2025 9:13 PM	Archivo CS	2 KB	
CameraController.cs.meta	11/25/2025 9:00 AM	Archivo META	1 KB	
DayNightCycle.cs	11/27/2025 9:35 PM	Archivo CS	0 KB	
DayNightCycle.cs.meta	11/27/2025 9:35 PM	Archivo META	1 KB	
MesaSync.cs	11/25/2025 9:00 AM	Archivo CS	10 KB	
MesaSync.cs.meta	11/25/2025 9:00 AM	Archivo META	1 KB	
UnityMainThreadDispatcher.cs	11/25/2025 9:00 AM	Archivo CS	1 KB	
UnityMainThreadDispatcher.cs.meta	11/25/2025 9:00 AM	Archivo META	1 KB	